Task 3 - MoE model (Deepseek)

INTRODUCTION

This task implements a scaled-down version of the DeepSeek Mixture-of-Experts (MoE) architecture, engineered to run strictly under 1 Billion total parameters. Developed entirely from scratch using JAX and Flax, this implementation bypasses heavy, high-level frameworks to isolate and analyze bare-metal hardware efficiency across CPU, GPU, and TPU backends. The primary objective of this implementation is to benchmark the throughput, token processing efficiency, and load-balancing stability of sparse architectures under constrained resource environments like Google Colab on CPU, GPU, and TPU.

THE INTUITION BEHIND DESIGN

In a traditional "Dense" model like Qwen or LLaMA, every single parameter is mathematically multiplied against every single word (token) that passes through the network. If we have a 1-Billion parameter model, processing a single word requires 1 billion calculations. On constrained hardware like a free Google Colab tier, this brute-force approach quickly hits a wall in both memory capacity and computation speed.

This DeepSeek-inspired Mixture-of-Experts (MoE) architecture solves this by completely decoupling the model's memory footprint from its computational footprint. Rather than pushing data through one massive Feed-Forward Network (FFN) at every layer, that massive network is sliced into 16 smaller, independent neural networks called **Experts**.

The Routing Mechanism

When a token enters a layer, it does not go to all 16 experts. Instead, it hits a **Router**—a small gating mechanism that evaluates the token and calculates a probability score for which experts are best suited to handle it. Based on this configuration, the router uses **Top-2 Routing** (top_k: 2), meaning it strictly sends the token to the two most relevant experts and completely ignores the other 14. This means the hardware is only performing math on a tiny fraction of the network at any given time, drastically speeding up the step time.

Shared vs. Routed Experts

Standard MoE models often suffer from a phenomenon where they lose general context because tokens are constantly scattered across different specialized experts. To fix this, the architecture implements **Shared Experts**.

- **Shared Experts (num_shared_experts: 2):** These two experts bypass the router entirely. Every single token is forced to pass through them. They act as the "general knowledge" foundation of the model, maintaining grammatical structure and core context.
- **Routed Experts (The remaining 14):** These handle specialized, highly specific feature extraction based on the router's decision.

The Parameter Breakdown (Active vs. Total)

By structuring the network this way, we successfully pack the knowledge capacity of a near-Billion parameter model into an incredibly lightweight computational loop.

- **Total Parameters (~900M):** The total memory footprint sitting in the GPU's VRAM. This represents the model's full capacity to learn and store information.
- **Active Parameters (~265M):** Because any given token only interacts with the Attention mechanism, the 2 Shared Experts, and 2 Routed Experts, the mathematical workload drops by nearly 70%. The hardware only computes math for ~265M parameters per step, delivering the speed of a tiny model with the intelligence of a much larger one.

SYSTEM ARCHITECTURE

config.yml (The Control Center)

- **Responsibility:** This is the single source of truth for all settings. It dictates the model's size (layers, experts), the training environment (batch size, steps), the dataset type (synthetic vs. real), and the hardware backend (CPU, GPU, or TPU).
- **Relationship:** It is completely passive. It does not execute any code. Instead, the other three Python files look at this document to know exactly how they should behave before they start running.

2. model.py (The Blueprint)

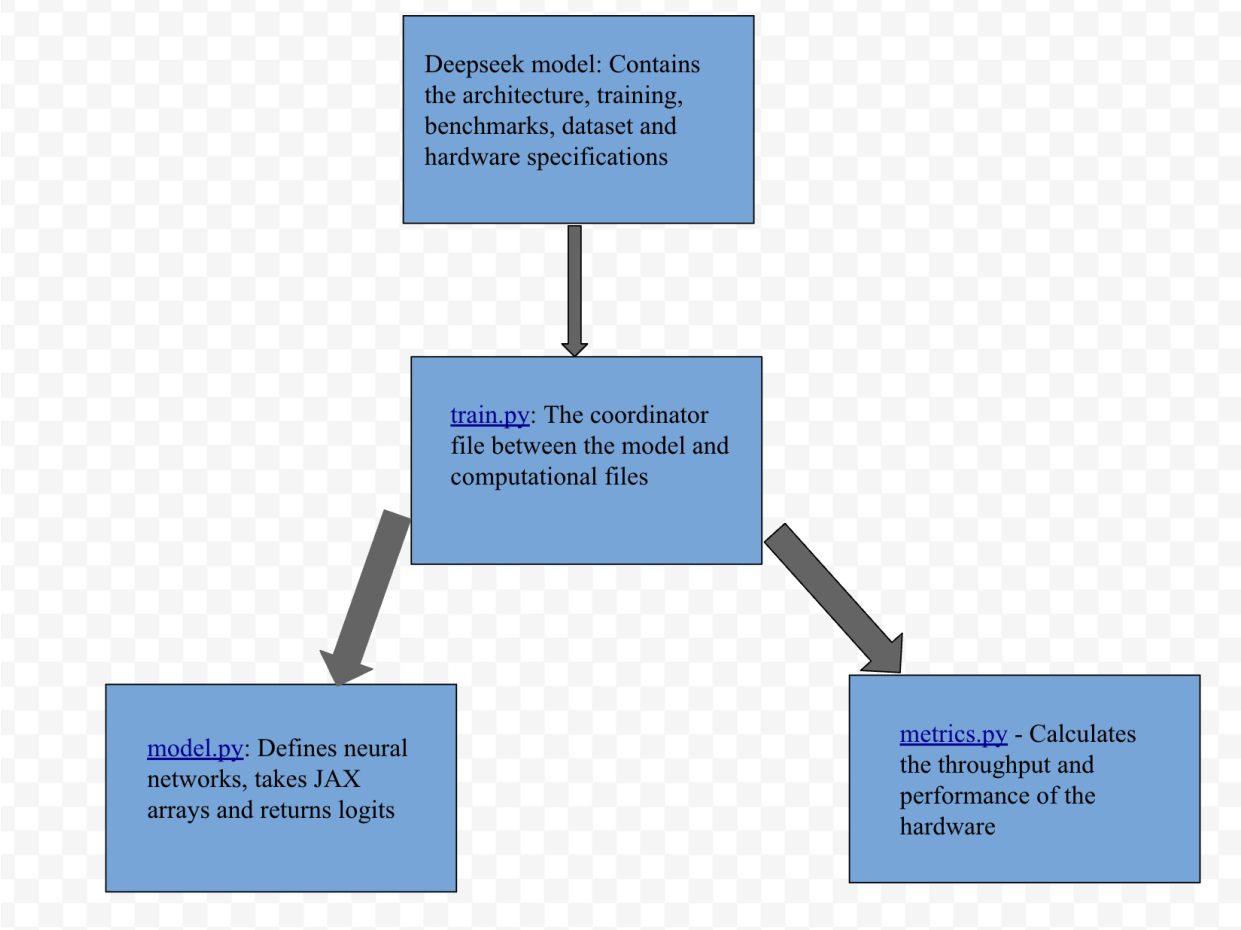
- **Responsibility:** This file defines the actual "brain" of the AI. Written entirely in pure Flax and JAX, it contains the mathematical blueprints for the neural network. This includes the Attention mechanism, the Mixture of Experts router, the shared and routed experts, and the Load Balancing calculation.
- **Relationship:** It relies on config.yml (via the MoEConfig dataclass) to know how big to build the network. It does not train itself; it merely provides the mathematical structure that train.py will use.

3. metrics.py (The Reporter)

- **Responsibility:** This file acts as the calculator for human-readable performance. While the neural network only understands raw math, this file takes the messy output and calculates throughput (Tokens per Second), formats the loss curves, and generates the final JSON report.
- **Relationship:** It sits quietly until the very end of the training process. Once train.py finishes its loop, it hands a massive dictionary of raw timestamps and loss numbers over to metrics.py to be cleaned up and saved.

4. train.py (The Engine)

- **Responsibility:** This is the main execution script. It handles the universal hardware routing (setting up the JAX compiler for our specific Colab hardware), loads the datasets (via Grain or synthetic generation), and runs the actual forward and backward mathematical passes to train the model.
- **Relationship:** This is the central hub. When we run train.py, it does the following in order:
    1. Reads the config.yml to understand the goal.
    2. Imports the neural network blueprint from model.py and initializes it in the hardware's memory.
    3. Runs the training loop, feeding data into the model step-by-step.
    4. Passes the final, raw results to metrics.py to generate the final performance report.



MATHEMATICAL CONCEPTS

Training a Mixture of Experts (MoE) requires a different mathematical approach than a standard dense model. Instead of optimizing just one goal (predicting the next word), the engine must optimize two conflicting goals simultaneously: **learning the language and managing the workload.**

1. The Gating Mechanism (Top-K Routing)

When a word (token) enters the MoE layer, the **Router** must decide which experts should process it. It does this by passing the token's data through a linear layer to generate a set of raw scores (logits) for all 14 routed experts.

It then applies a Softmax function to turn these scores into probabilities (percentages that add up to 100%). Finally, it applies a top_k operation to select only the two experts with the highest probabilities, multiplying the token's data by these probability weights before passing it inside.

2. The Dual-Loss System

To train this network, two separate losses are calculated and then added together:

A. The Task Loss (Cross-Entropy):

This is the standard loss used in all Language Models. It calculates how wrong the model's prediction was compared to the actual next word in the sentence. The standard Softmax Cross-Entropy is used for this calculation.

B. The Load Balancing Loss (Auxiliary Loss):

Neural networks naturally look for the easiest path. Without intervention, the Router will quickly decide that "Expert 1" is good at everything and route 100% of the tokens there, leaving the other 13 experts completely dead. This is called Expert Collapse.

To prevent this, the model is mathematically penalized if it plays favorites or takes the easy path. The load balancing loss (L_balance) ensures tokens are distributed evenly. It is calculated using this formula:

$$L_{balance} = N \sum_{i=1}^{N} f_i \cdot P_i$$

Where:

- **N** = The total number of routed experts (14 in this case).
- **fi** = The actual fraction of tokens sent to expert i in this batch.
- **pi** = The router's average probability assigned to expert i.

MEASURES TAKEN TO SCALE DOWN THE MODEL

1. Crushed the Expert Intermediate Size (1024)

- **The change:** Standard dense models (like the 0.8B Qwen) use an intermediate feed-forward size of roughly 4096. In this configuration, the expert_intermediate_size is reduced to 1024.
- **The reason:** Because 16 experts per layer were instantiated , retaining an intermediate size of 4096 would multiply out to massive VRAM requirements. By shrinking the internal dimension of the experts, a "narrower" but highly specialized individual neural networks were created, enabling the model to afford 16 of them without causing an Out-Of-Memory (OOM) error.

2. Reduced the Global Hidden Size (1024)

- **The change:** The core hidden_size of the model was reduced to 1024 (down from the typical 1536 or 2048 seen in 1B class models).
- **The reason:** The hidden_size dictates the mathematical width of the entire model, including the Attention mechanism and the input/output boundaries of the MoE layer. Shrinking this global dimension compresses the size of the attention matrices (Q, K, V) globally, freeing up vital memory budgets to spend on instantiating the routing mechanism and shared experts instead.

3. Trimmed Layer Depth (16 Layers)

- **The change:** The total number of Transformer blocks (num_layers) were reduced from the standard 24 down to 16.
- **The reason:** Transformer models scale linearly with depth. A 16-expert MoE layer is incredibly parameter-dense. Capping the depth at 16 layers kept the total parameter count safely below 900M, ensuring the AdamW optimizer (which triples memory requirements during training to store momentum and variance states) could fit entirely inside the free 15GB T4 GPU.

4. Implemented Grouped-Query Attention (GQA)

- **The change:** Instead of matching the 16 Attention Heads with 16 Key/Value (KV) heads, the Key/Value heads were restricted to 8 (num_key_value_heads: 8).
- **The reason:** Grouped-Query Attention forces multiple Query heads to share the same Key/Value matrices. This halves the parameter size of the K and V projections in the attention layer, further recovering memory that the massive expert arrays demanded.

All the config files are uploaded in the configs folder only with synthetic data as instructed in the assignment:

1. Deepseek0.9b for CPU
2. Deepseek0.9b for GPU
3. Deepseek0.9b for TPU

RESULTS

The outputs of every run of the model on CPU, GPU, and TPU is logged and stored in the outputs folder by calculating all the metrics through a separate Python script:

1. Deepseek0.9b_cpu_benchmarking
2. Deepseek0.9b_gpu_benchmarking
3. deepseek0.9b_tpu_benchmarking

METRIC	CPU	GPU	TPU
Tokens/step	512	512	512
Memory footprint	~3.6 GB (System RAM)	~3.6 GB (VRAM)	~3.6 GB (PER MXU CORE)
Throughput	~121.9	~1422.2	~ 40960

DIFFERENCE BETWEEN BOTH THE MODELS

The core difference between a dense model like Qwen and an MoE model like DeepSeek comes down to how they spend their computing power. In a dense model, every single artificial "neuron" activates for every single word we type, which requires massive computational heavy lifting and severely slows down the hardware. An MoE model, on the other hand, works like a large company with a smart receptionist; instead of sending a task to every employee, a routing mechanism evaluates the word and sends it only to a few highly specialized "experts." This is why the MoE model runs exponentially faster on the GPU and survives the CPU without crashing. Even though it stores almost a billion parameters in its memory to retain its overall

knowledge, it only actively uses a small fraction of them to perform the math for any given word, effectively decoupling memory capacity from raw computational speed.

NOTE:

This task was implemented in the same environment with the same tech stack as the previous task. Hence, the precautions taken in the earlier task to avoid all the problems listed in the document - Task 2 is applicable for this implementation as well. The folder architecture and the splitting of files' responsibilities is also similar to the previous task.